

SPACE CADET

Бизнес 2: Получение доступа к прошивке современной компьютерной платы с использованием различных техник реверс-инжиниринга



О команде

Капитан: Савельев Андрей Вениаминович, backend

Кол-во участников: 5

Краткое описание:

Команда Space Cadet - постоянный участник и призер хактонов и CTF-соревнований, финалист ЛЦТ-2025. ВКА им. А.Ф.Можайского.

Город и регион: Санкт-Петербург

SPACE CADET

Краткое описание решения:

Мы полностью разобрали USB-устройство CyberSafe от дампа прошивки до расшифровки хранилища. Нашли пароль в открытом виде через IDA Pro. Пропатчили прошивку в двух вариантах: одна принимает любой код, вторая разблокирует устройство сразу без ввода. Разобрали алгоритм шифрования хранилища и расшифровали все хранилище данных. Сделали аппаратный и программный bruteforce, а также нашли side-channel вектор атаки, связанный с измерением времени работы светодиодов.

Уникальность решения:

Семь полноценных рабочих векторов атаки, покрывающих все основные классы атак на физические устройства: статический анализ, модификация прошивки, криптоанализ, аппаратный bruteforce, эмуляция, side-channel и физическая модификация.



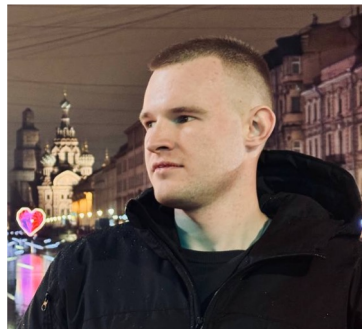
**Елизар
Бровко**

- Penetration Tester
- @Nortonsa
- +7 (988) 866-07-99
- ВКА



**Андрей
Савельев**

- Backend Developer
- @srDroopy
- +7 (995) 987-63-30
- ВКА



**Сергей
Климов**

- Hardware Reverse Engineer
- @nopowerbnk
- +7 (914) 571-12-69
- ВКА



**Алексей
Глухов**

Reverse Engineer
@ alexeyzxc
+7 (981) 915-76-42
ВКА



**Илья
Полькин**

Reverse Engineer
@ Shakkkall

- 7 (909) 841-70-88
- ВКА

Краткая история команды:

01

Команда Space Cadet собрана на базе Военно-космической академии имени А.Ф. Можайского. Участники, финалисты и призёры CTF-соревнований и Хакатонов. Мы продолжаем соревновательный путь выпускников – перенимаем их опыт и не сбавляем темп.

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

02

Нам интересен реверс-инжиниринг железа, а не только софта. Эта задача даёт редкую возможность поработать с реальным устройством: от дампа прошивки до расшифровки хранилища. Здесь можно применить весь набор навыков команды – от статического анализа до работы с GPIO. И это не учебный пример, а реальный продукт со своими слабостями.

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

03

Основная сложность заключалась в том, что устройство не взаимодействует по USB до разблокировки – это исключило стандартные методы анализа. Дополнительно отсутствовала какая-либо документация, электрические схемы и исходные коды прошивки – устройство исследовалось как «чёрный ящик». Мы обошли это через дампы прошивки в режиме BOOTSEL и последующий анализ.

Техническая суть решения

Устройство исследовалось как «черный ящик» без документации, электрических схем и исходного кода прошивки.

Мы прошли полный цикл: получили дампы памяти через режим BOOTSEL, восстановили логику работы в IDA Pro, обошли проверку пароля и расшифровали виртуальное хранилище.

Работа велась на стыке аппаратного и программного анализа – от пайки и работы с GPIO до криптоанализа и эмуляции.

1. Подготовительный этап;

2. Векторы атаки:

- извлечение пароля;
- модификация прошивки;
- криптоанализ хранилища;
- эмуляция функций прошивки;
- аппаратный перебор и timing-атака;
- выпаивание чипа памяти.

Результат: реализовано шесть полноценных векторов атаки, покрывающих все основные классы – от статического анализа и модификации прошивки до криптоанализа, аппаратного перебора и side-channel. Показано, что защита устройства держится на слабом шифре, хранении пароля в открытом виде и отсутствии блокировки попыток.

ДАМП ПАМЯТИ

Удалось получить полный дамп флеш-памяти устройства через встроенный режим BOOTSEL, который есть в RP2040-Zero.

Что было сделано:

1. Перевели плату в режим BOOTSEL. Зажали кнопку BOOT на плате и подали питание через USB. Плата определилась как диск RPI-RP2 вместо обычного режима работы.
2. Считали дамп через picotool. (picotool save -a firmware.bin)

В RP2040-Zero есть встроенный boot ROM, который выполняется до основной прошивки. При зажатой кнопке BOOT чип переходит в режим программирования и открывает доступ к флеш-памяти. Этот механизм нельзя отключить – у RP2040-Zero нет OTP-блокировки или secure boot fuse.

РАЗБОР СТРУКТУРЫ ПАМЯТИ

Из открытой документации платы RP2040-Zero: процессор двухъядерный ARM Cortex-M0+; встроенная флэш-память составляет 2 МБ (ровно как и наш дамп).

Для правильного отображения кода в IDA Pro необходимо выставить тип процессора ARM Little-endian (рис. 1), а также для корректного отображения реальных адресов памяти, начальный адрес Read-Only Memory и адрес загрузки выставляем в 0x10000000 (рис. 2).

После автоматического анализа IDA Pro корректно дизассемблировала прошивку и восстановила структуру кода (рис. 3). Это дало нам полную картину работы устройства и возможность искать уязвимости.

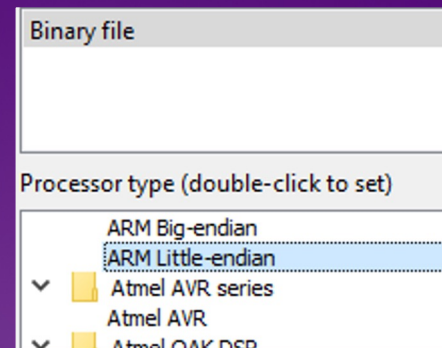


Рис. 1

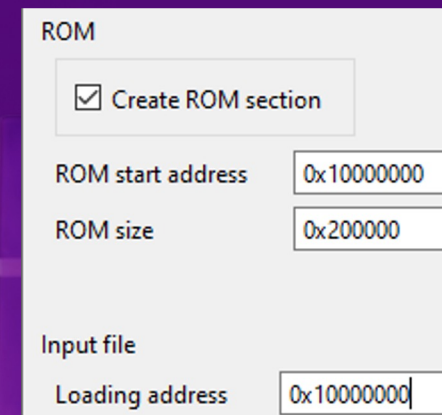


Рис. 2

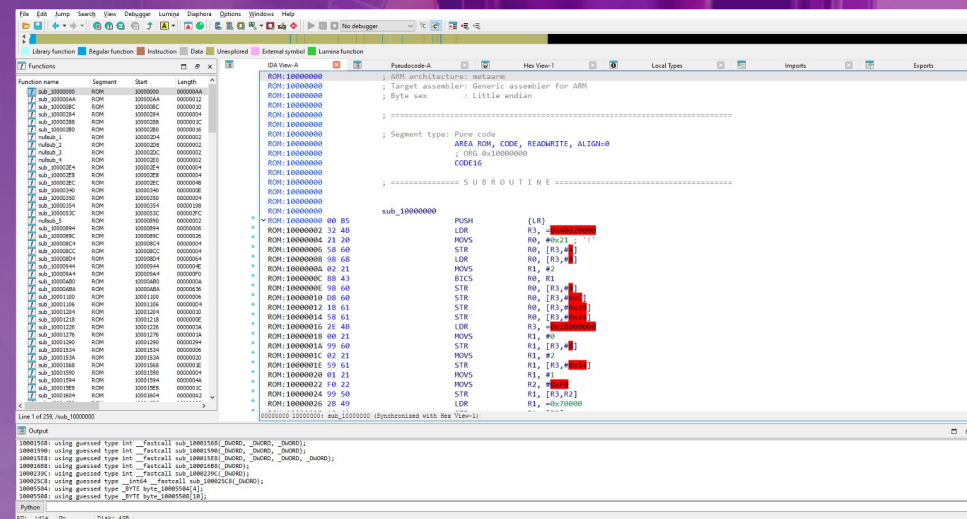


Рис. 3

ИЗВЛЕЧЕНИЕ ПАРОЛЯ ИЗ ПРОШИВКИ

Отталкивались от векторной таблицы на 0x10000000 нашли Reset Handler, первую функцию после включения. По цепочке вызовов от него дошли до sub_1000089C: она помечена как __noreturn, содержит бесконечный цикл while(1) и инициализацию периферии перед ним (рис. 4) – это классические признаки main во встраиваемых системах.

Внутри главного цикла main вызывается sub_10000AB0, которая сразу передаёт управление в sub_10000ABA. Эта функция работает с GPIO энкодера, светодиодами, хранит введённые цифры в буфере и сравнивает их с эталоном – здесь и находится логика проверки PIN-кода.

Внутри sub_10000ABA нашли цикл сравнения введённых цифр с эталоном. По адресу 0x100054FF и лежали верные цифры пароля (рис. 6).

```
void __noreturn sub_1000089C()
{
    int v0; // r0
    int v1; // r0
    int v2; // r0

    v0 = sub_100050E0();
    v1 = sub_1000305C(v0);
    sub_1000AB0(v1);
    sub_10004E64(0, 0);
    while ( 1 )
    {
        v2 = sub_10003858(-1, 0);
        sub_100009A4(v2);
    }
}
```

Рис. 4

ROM:100054FF	00	DCB	0
ROM:10005500	06	DCB	6
ROM:10005501	07	DCB	7
ROM:10005502	01	DCB	1
ROM:10005503	00	DCB	0

Рис. 6

```
v20 = 1;
*(v17 + 0x20001EB0) = 1;
while ( *(&v37 + v20) == *(v20 + 0x100054FF) )
{
    ++v20;
    sub_1000239C(50);
    if ( v20 == 5 )
    {
        sub_10001218(0, 32, 0);
        MEMORY[0xD0000014] = 1024;
        MEMORY[0x20002ED2] = 1;
        __asm { POP {R4-R7,PC} }
    }
}
```

Рис. 5

1-й Способ

Подключение по любому паролю

Этот вариант подразумевает собой патчинг сравнения введенной и эталонной цифры, т.е. чтобы вне зависимости от равенства, введенная цифра считалась бы правильной.

Этого удалось достичь путем создания безусловного перехода.

Было:

```
05 2C      CMP      R4, #5
F3 D1      BNE      loc_10000C82
```

Это условный переход, если все 4 цифры введены правильно, то устройство разблокируется.

Стало:

```
02 2C      CMP      R4, #2
C0 46      NOP
```

Это позволяет даже при вводе 4 неправильных цифр пройти к коду, который разблокирует виртуальное хранилище и подключает его к ПК.

2-й Способ

Подключение без пароля

В первую очередь необходимо избавиться от проверки на ввод 4 цифр с энкодера. К тем изменениям что уже были, мы убираем проверку длины введенных цифр.

Было:

```
02 2E      CMP      R6, #2
00 D8      BHI      loc_10000C78
CF E0      B        loc_10000E18
```

Это условный переход, если введено больше двух цифр, переходим к проверке пароля.

Стало:

```
02 2E      CMP      R6, #2
00 E0      B        loc_10000C78
CF E0      B        loc_10000E18
```

Это одно из изменений, но не единственное. В коде есть ещё несколько небольших правок, которые в совокупности дают возможность разблокировки без пароля.

Проанализировав дизассемблированный код, видно что существуют две функции, которые шифруют/расшифровывают данные с помощью XOR (sub_1000053C/sub_10000354 соответственно). Т.к. это XOR, то для расшифрования, можно провести еще один раз функцию шифрования, в связи с тем, что XOR – обратимый процесс.

Это видно по псевдокоду: обе функции реализуют один и тот же алгоритм, отличаются только точкой вызова. Слева представлена функция расшифровки, а справа шифрования (Рис. 7).

На основе анализа псевдокода был написан Python-скрипт для расшифровки виртуального хранилища.

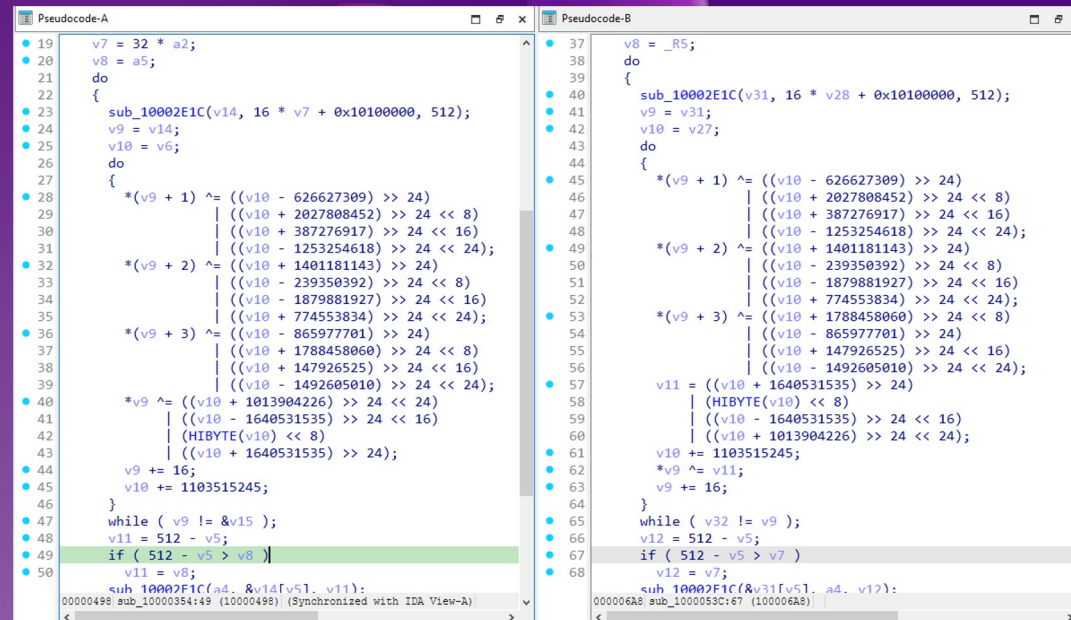


Рис. 7

ЭМУЛЯЦИЯ ПРОШИВКИ

Для работы с прошивкой без физического токена была восстановлена необходимая среда исполнения в **Unicorn Engine**: память Flash/RAM, периферийные регистры и SPI Flash.

В эмуляторе запускается оригинальная функция проверки PIN (**0x0C78**). Для каждой комбинации 0000–9999 изменяются входные данные в памяти, после чего выполняется машинный код прошивки. Найденный PIN подтверждается самой прошивкой.

Полученный подход также позволил вызвать оригинальную функцию расшифровки (**0x354**) и восстановить зашифрованное хранилище в виде FAT12-образа (Рис. 8).

```
try:
    (constant) DECRYPT_FN: Literal[268436308] = FLASH_BASE + 0x354
    self.uc.emu_start(DECRYPT_FN | 1, 0, count=1000000, timeout=0)
```

Рис. 8

```
[crack] success at 7917 (stop at 0x10000cc6)
PIN = 7917
[*] decrypting the 1MB storage region -> token_disk.img
disk image written: /Users/elizavr/Desktop/hack/token_disk.img
[*] attaching as a virtual disk...
/dev/disk5                                /Volumes/NO NAME
```

Рис. 9



Flash
dump

Unicorn

эмуляция
окружения

оригинальный
код

PIN /
расшиф
ровка

АППАРАТНЫЙ BRUTEFORCE

Один из ключевых векторов - прямой перебор PIN-кода. В прошивке нет никакой защиты от повторных попыток: нет счетчика неудачных вводов, нет блокировки после N ошибок, нет задержки между попытками. Устройство после каждого неверного кода просто сбрасывает ввод и снова готово принимать. Это делает перебор всех 10 000 комбинаций полностью реализуемым - вопрос только во времени.

Мы автоматизировали процесс через второй микроконтроллер (рис. 10), который эмулирует энкодер. Подключились к линиям вращения и нажатия кнопки напрямую, без физического воздействия на сам энкодер. В процессе первой попытки плата вышла из строя - сказала разница потенциалов и обратный ток со стороны контроллера замка. Заменили её и продолжили работу. В прошивку для демонстрации зашили приоритетный код 0007, чтобы не ждать полного перебора.

Тестирование подтвердило работоспособность метода. Устройство разблокировалось, хранилище стало доступно. Главный вывод - отсутствие блокировки после неудачных попыток делает полный перебор реализуемым. Для атаки достаточно физического доступа к линиям энкодера, разборка устройства и чтение прошивки не требуются.

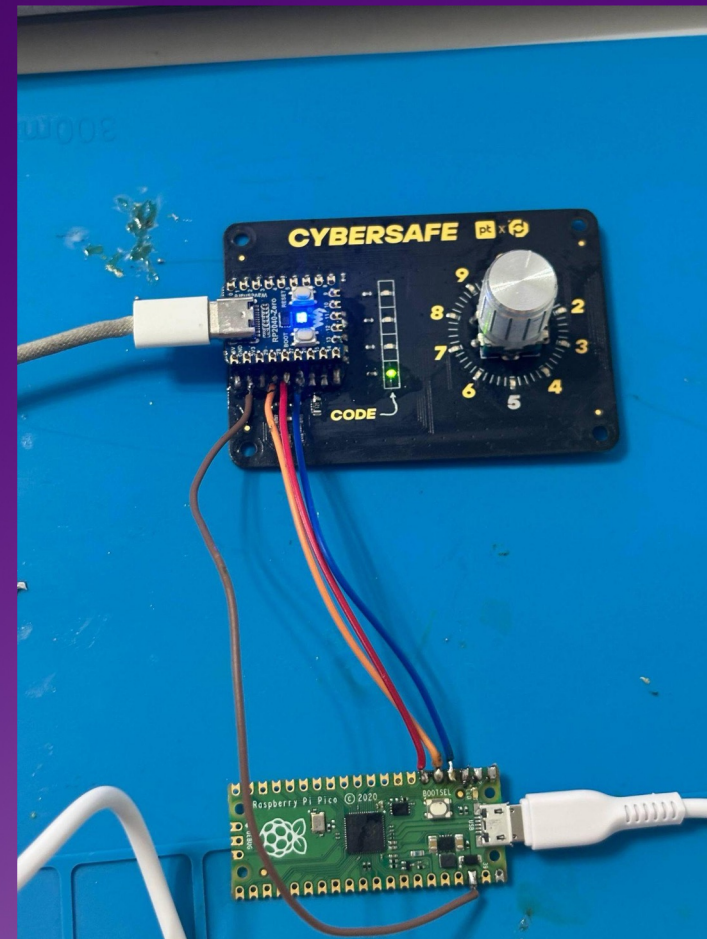


Рис. 10

SIDE-CHANNEL АТАКА

В прошивке проверка пароля реализована небезопасно с точки зрения времени. Устройство сравнивает цифры по одной и прерывается на первой ошибке, но тратит фиксированную задержку на каждую совпавшую цифру. Из-за этого время отклика линейно растёт: чем больше ведущих цифр угадано, тем дольше устройство думает перед тем, как показать ошибку. Мы измерили эту разницу - примерно 50 мс на каждую верную цифру.

Для замера использовали логический анализатор. Первый канал подключили к линии кнопки, второй - к управляющему выводу индикаторной лампочки (рис. 11). В программе анализатора настроили курсор для замера разницы времени между нажатием и реакцией устройства. Эмпирически зафиксировали пороги: около 1.20 с при неверной первой цифре, 1.25 с при верной первой и неверной второй, 1.30 с при верных первых двух, 1.35 с при верных первых трех (рис. 12-15).

Это позволяет подбирать код не целиком, а по одной цифре. На каждом шаге фиксируем уже найденные цифры и перебираем следующую, ориентируясь на скачок времени отклика. Пространство перебора сокращается с 10 000 комбинаций до 40 попыток.



Рис. 11

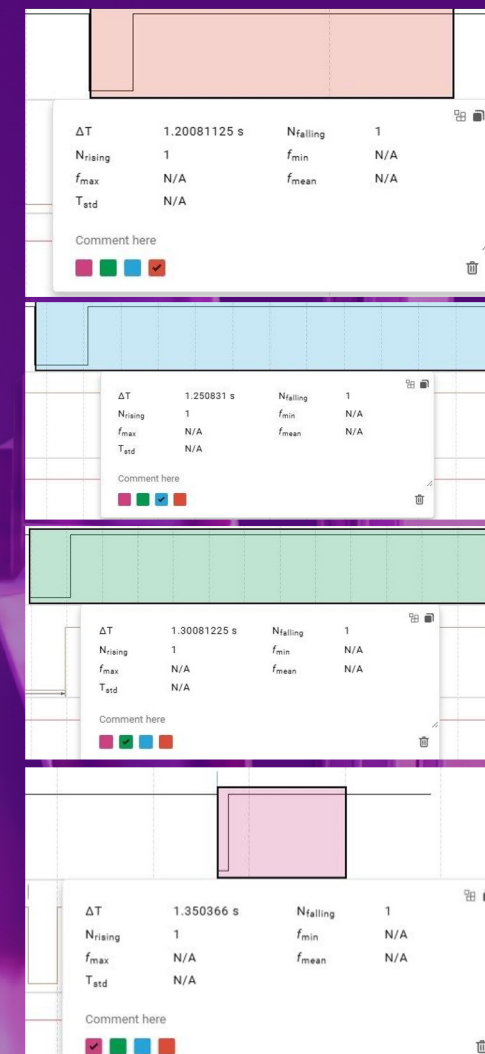


Рис. 12-15

ВЫПАИВАНИЯ ЧИПА ПАМЯТИ

Тут текст



ИТОГ

